HPC1

1. Design and implement Parallel Breadth First Search and Depth First Search based on existing

algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS .

```
#include <iostream>     // for input-output
#include <vector>       // for adjacency list
#include <queue>        // for BFS queue
#include <omp.h>        // for OpenMP parallelization
#include <algorithm>    // for fill()
using namespace std;

// Adjacency list representation of graph
vector<int> graph[100];

// Visited array to track visited nodes
bool visited[100];

// 🔵 Parallel BFS Function
void bfs(int start) {
    queue<int> q;              // queue for BFS
    visited[start] = true;     // mark starting node as visited
    q.push(start);             // push start node into queue

    cout << "BFS Traversal: ";

    // Loop until queue becomes empty
    while (!q.empty()) {
        int node = q.front();  // get front element
```

```
        q.pop();              // remove it from queue

        cout << node << " ";   // print node

        // Parallel loop to process all neighbors
        #pragma omp parallel for
        for (int i = 0; i < graph[node].size(); i++) {
            int neighbor = graph[node][i];  // get neighbor

            // If neighbor is not visited
            if (!visited[neighbor]) {

                // Critical section to avoid race condition
                #pragma omp critical
                {
                    if (!visited[neighbor]) {   // check again inside critical
                        visited[neighbor] = true; // mark visited
                        q.push(neighbor);       // push into queue
                    }
                }
            }
        }
    }
    cout << endl;
}

// 🔴 Parallel DFS Function
void dfs(int node) {
    visited[node] = true;      // mark node as visited
```

```
    cout << node << " ";       // print node

    // Parallel loop for neighbors
    #pragma omp parallel for
    for (int i = 0; i < graph[node].size(); i++) {
        int neighbor = graph[node][i];  // get neighbor

        // If not visited, call DFS recursively
        if (!visited[neighbor]) {
            dfs(neighbor);   // recursive DFS call
        }
    }
}

// 🔹 Main Function
int main() {
    int n, e;

    cout << "Enter number of nodes: ";
    cin >> n;   // input number of nodes

    cout << "Enter number of edges: ";
    cin >> e;   // input number of edges

    cout << "Enter edges (u v):\n";

    // Input edges
    for (int i = 0; i < e; i++) {
        int u, v;
```

```
        cin >> u >> v;

        // Since graph is undirected, add both ways
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    int startNode;
    cout << "Enter starting node: ";
    cin >> startNode;   // input starting node

    // Initialize visited array to false
    fill(visited, visited + n, false);

    bfs(startNode);     // call BFS

    // Reset visited array for DFS
    fill(visited, visited + n, false);

    cout << "DFS Traversal: ";
    dfs(startNode);     // call DFS

    cout << endl;

    return 0;
}
```

📌 Given Graph (Example)

```
   0
  / \
 1   2
/ \   \
3   4   5
```

Edges:

0-1, 0-2, 1-3, 1-4, 2-5

Start node = **0**

🔵 BFS Dry Run (Step-by-Step)

Initial State:

visited = [false false false false false false]
queue = []

Step 1:

start = 0
visited[0] = true
queue = [0]

Step 2: Process node 0

queue = [0]
pop → 0
print → 0

Neighbors of 0 → (1, 2)

👉 Parallel loop runs for both neighbors

- Visit 1 → mark visited → push
- Visit 2 → mark visited → push

visited = [true true true false false false]
queue = [1, 2]

Step 3: Process node 1

pop → 1
print → 1

Neighbors → (0, 3, 4)

- 0 already visited ❌
- 3 → mark + push ✓
- 4 → mark + push ✓

visited = [true true true true true false]
queue = [2, 3, 4]

Step 4: Process node 2

pop → 2
print → 2

Neighbors → (0, 5)

- 0 already visited ❌
- 5 → mark + push ✓

visited = [true true true true true true]
queue = [3, 4, 5]

Step 5:

pop → 3 → print
pop → 4 → print
pop → 5 → print

✅ Final BFS Output:

0 1 2 3 4 5

🔴 DFS Dry Run (Step-by-Step)

Initial:

visited = [false false false false false false]

Step 1:

dfs(0)
visited[0] = true
print → 0

Step 2: Go to neighbor 1

dfs(1)
visited[1] = true
print → 1

Step 3: Go to neighbor 3

dfs(3)
visited[3] = true
print → 3

Backtrack to 1

Step 4: Next neighbor 4

dfs(4)
visited[4] = true
print → 4

Backtrack to 1 → then 0

Step 5: Go to neighbor 2

dfs(2)
visited[2] = true
print → 2

Step 6: Go to neighbor 5

dfs(5)
visited[5] = true
print → 5

✅ **Final DFS Output:**

0 1 3 4 2 5

⚠️ In parallel version, order may change slightly.

⚙️ How Parallelism Works Here

🔵 BFS

#pragma omp parallel for

👉 Runs neighbor loop in parallel
👉 Multiple threads process neighbors (1 and 2 together)

🔴 DFS

#pragma omp parallel for

👉 Recursive calls for neighbors run in parallel
👉 Different branches explored simultaneously

🔐 Why critical is Used

#pragma omp critical

👉 Prevents:

- Two threads modifying visited[]
- Two threads pushing into queue at same time

🎯 One-Line Summary (Say in Viva)

BFS:

"Nodes are processed level-wise, and neighbors are explored in parallel."

DFS:

"Recursive calls are parallelized to explore different branches simultaneously."

⚠️ Important Viva Point

"Due to shared structures like queue and visited array, synchronization is required, so full parallelism is not achieved."

🧠 **Pro Tip (Exam Booster)**

If you draw this:

Queue evolution (BFS):
[0] → [1,2] → [2,3,4] → [3,4,5]

👉 You instantly impress examiner.

📘 Program Explanation (Line by Line)

🔹 Header Files

#include <iostream>

👉 Used for input/output (cin, cout)

#include <vector>

👉 Used to store adjacency list (graph)

#include <queue>

👉 Required for BFS traversal

#include <omp.h>

👉 Enables OpenMP (parallel programming)

#include <algorithm>

👉 Used for fill() function

🔹 Namespace

using namespace std;

👉 Avoid writing std:: repeatedly

🔹 Global Variables

vector<int> graph[100];

👉 Adjacency list for graph
👉 Each index stores neighbors of that node

bool visited[100];

👉 Keeps track of visited nodes
👉 Prevents revisiting same node

🔵 BFS Function

void bfs(int start)

👉 Function to perform Breadth First Search

```
queue<int> q;
```

👉 Queue used for BFS traversal

```
visited[start] = true;
q.push(start);
```

👉 Mark start node as visited
👉 Insert it into queue

```
cout << "BFS Traversal: ";
```

👉 Print heading

```
while (!q.empty())
```

👉 Loop runs until queue is empty

```
int node = q.front();
q.pop();
```

👉 Take front node from queue
👉 Remove it

```
cout << node << " ";
```

👉 Print current node

🔥 **Parallel Part**

```
#pragma omp parallel for
```

👉 Parallelizes loop
👉 Multiple threads process neighbors simultaneously

```
for (int i = 0; i < graph[node].size(); i++)
```

👉 Loop through all neighbors of current node

```
int neighbor = graph[node][i];
```

👉 Get neighbor node

```
if (!visited[neighbor])
```

👉 Check if neighbor is not visited

```
#pragma omp critical
```

👉 Critical section ensures:

- Only one thread updates shared data at a time

```
if (!visited[neighbor]) {
    visited[neighbor] = true;
    q.push(neighbor);
}
```

👉 Mark neighbor visited
👉 Add it to queue

```
cout << endl;
```

👉 End BFS output

🔴 **DFS Function**

```
void dfs(int node)
```

👉 Recursive function for Depth First Search

```
visited[node] = true;
cout << node << " ";
```

👉 Mark node visited
👉 Print node

#pragma omp parallel for

👉 Parallelize neighbor traversal

---

for (int i = 0; i < graph[node].size(); i++)

👉 Loop through neighbors

---

int neighbor = graph[node][i];

👉 Get neighbor

---

if (!visited[neighbor])

👉 Check if not visited

---

dfs(neighbor);

👉 Recursive call for DFS

⚠️ **Important Note (Say This in Viva)**

👉 *"DFS recursion is parallelized, but due to shared visited array, full parallelism is not achieved."*

🔹 **Main Function**

int main()

👉 Program starts here

---

int n, e;

👉 n = number of nodes
👉 e = number of edges

```
cout << "Enter number of nodes: ";
cin >> n;
```

👉 Take number of nodes

```
cout << "Enter number of edges: ";
cin >> e;
```

👉 Take number of edges

```
for (int i = 0; i < e; i++)
```

👉 Loop to input edges

```
cin >> u >> v;
```

👉 Input edge

```
graph[u].push_back(v);
graph[v].push_back(u);
```

👉 Store graph (undirected)

```
cout << "Enter starting node: ";
cin >> startNode;
```

👉 Input starting node

```
fill(visited, visited + n, false);
```

👉 Initialize all nodes as unvisited

```
bfs(startNode);
```

👉 Call BFS

---

fill(visited, visited + n, false);

👉 Reset visited array

---

cout << "DFS Traversal: ";
dfs(startNode);

👉 Call DFS

---

return 0;

👉 End program

🎯 Final 3-Line Explanation (Say This 💯)

“Graph is stored using adjacency list. BFS uses queue and processes neighbors in parallel using OpenMP. DFS uses recursion and parallelizes neighbor traversal, but synchronization is required due to shared visited array.”

🔥 Bonus Viva Line (Very Impressive)

“Parallelism improves performance for large graphs, but synchronization overhead limits speedup.”

▶️ How to Compile & Run (IMPORTANT)

Make sure you use OpenMP flag:

```
g++ yourfile.cpp -fopenmp -o output
./output
```

🧪 Example Run (What You Will See)

Input:

```
Enter number of nodes: 6
Enter number of edges: 5
Enter edges (u v):
0 1
0 2
1 3
1 4
2 5
Enter starting node: 0
```

✅ Expected Output

```
BFS Traversal: 0 1 2 3 4 5
DFS Traversal: 0 1 3 4 2 5
```

⚠️ VERY IMPORTANT (Don't Panic)

1. DFS Output May Change

You might get:

```
DFS Traversal: 0 2 5 1 4 3
```

👉 This is **CORRECT**
Reason:

- DFS is parallel
- Order is **not fixed**

---

2. Output Order Might Look Messy (Rare)

Example:

BFS Traversal: 0 2 1 3 5 4

👉 Still correct
👉 Parallel threads change order

🚨 **If You Get Wrong Output**

❌ **Case: Duplicate nodes**

Example:

0 1 2 2 3 4

👉 Cause:

- Race condition

👉 Fix:

- Make sure #pragma omp critical is present

❌ **Case: Program crash**

👉 Check:

- Node values should be < 100
- Input edges should be valid

🎯 **What To Tell Examiner After Running**

👉 Say confidently:

"Due to parallel execution, traversal order may vary, but all nodes are correctly visited once."

🎯 Core Viva Questions (Must Prepare)

❓ 1. What is BFS?

Answer:
Breadth First Search (BFS) is a graph traversal algorithm that explores nodes **level by level** using a **queue** data structure.

❓ 2. What is DFS?

Answer:
Depth First Search (DFS) is a graph traversal algorithm that explores nodes **depth-wise** using **recursion or stack**.

❓ 3. Difference between BFS and DFS?

BFS	DFS
Level-wise traversal	Depth-wise traversal
Uses queue	Uses recursion/stack
Finds shortest path	Does not guarantee shortest path

❓ 4. What is OpenMP?

Answer:
OpenMP is an API used for **parallel programming in C/C++** that allows execution using multiple threads on shared memory systems.

❓ 5. What does #pragma omp parallel for do?

Answer:
It divides loop iterations among multiple threads so they execute **simultaneously**.

❓ 6. Why is visited[] array used?

Answer:
To ensure each node is visited only once and to avoid infinite loops in cyclic graphs.

❓ 7. Why do we use #pragma omp critical?

Answer:
To prevent **race conditions** when multiple threads try to modify shared data like visited[] or queue.

❓ 8. What is a race condition?

Answer:
A race condition occurs when multiple threads access and modify shared data at the same time, leading to incorrect results.

❓ 9. Is your BFS fully parallel?

Answer (important):
No. It is **partially parallel** because the queue is shared and requires synchronization, limiting full parallelism.

❓ 10. Is DFS output always same?

Answer:
No. In parallel DFS, the output order may vary because multiple branches are executed simultaneously.

⚙️ Code-Based Questions

❓ 11. Why is queue used in BFS?

Answer:
Because BFS follows **FIFO (First In First Out)** to process nodes level by level.

❓ 12. Why recursion is used in DFS?

Answer:
Recursion naturally follows the **depth-first approach**, exploring one branch completely before backtracking.

❓ 13. Why do we reset visited[] before DFS?

Answer:
Because BFS already marked nodes visited, so we reset to allow DFS to traverse all nodes again.

❓ 14. What happens if we remove critical section?

Answer:
It may cause:

- Duplicate nodes
- Incorrect traversal
- Program crash due to race condition

❓ 15. Can BFS be implemented without queue?

Answer:
No. Queue is essential for maintaining level-wise traversal order.

🚀 Conceptual Questions (Very Important)

❓ 16. Why is BFS difficult to parallelize?

Answer:
Because BFS depends on **level-wise order**, and nodes must be processed in sequence using a shared queue.

❓ 17. What is synchronization overhead?

Answer:
Extra time required to manage shared resources (like locks/critical sections), which reduces performance.

❓ 18. When is parallel BFS useful?

Answer:
For **large graphs**, where multiple nodes at the same level can be processed simultaneously.

❓ 19. What are advantages of parallel computing?

Answer:

- Faster execution
- Better CPU utilization
- Handles large-scale problems efficiently

❓ 20. Limitations of your program?

Answer:

- Not fully parallel
- Synchronization overhead
- Output order may vary
- Less benefit for small graphs

💡 Smart Extra Questions (High Chance)

❓ 21. What is shared memory model?

Answer:
A system where all threads access the same memory space, like in OpenMP.

❓ 22. What is thread?

Answer:
A lightweight unit of execution within a process.

❓ 23. What is parallelism vs concurrency?

Answer:

- Parallelism → tasks run at same time (multiple cores)
- Concurrency → tasks appear simultaneous but may share CPU

❓ 24. How many threads OpenMP uses?

Answer:
By default, it uses number of CPU cores, but it can be controlled using:

```
export OMP_NUM_THREADS=4
```